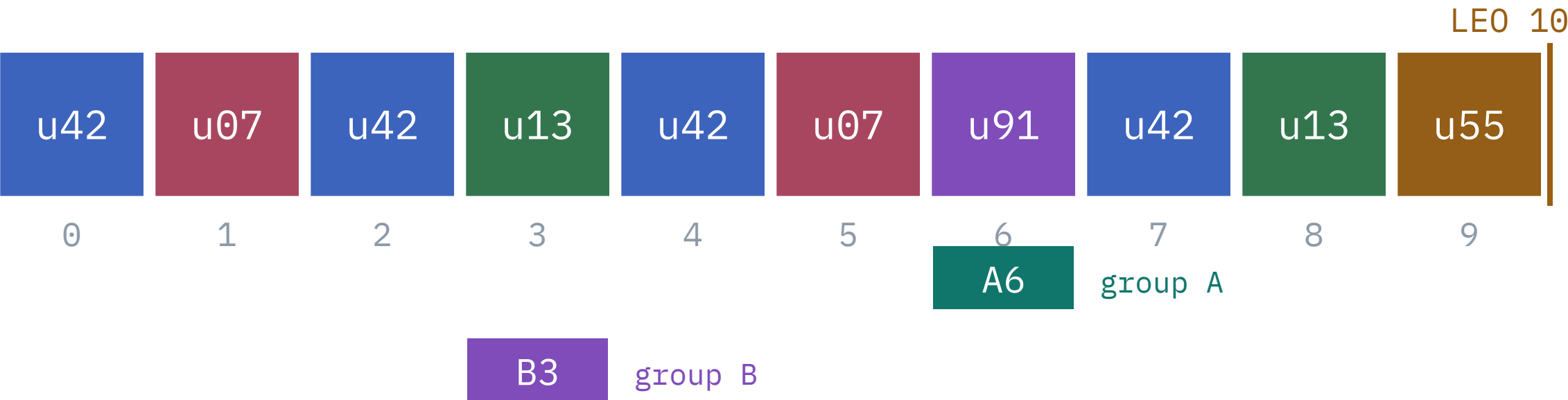


FOR EVERYONE KAFKA NEVER CLICKED FOR

# Kafka is not a queue.

And while you keep a queue in your head, nothing adds up: not replay, not rebalances, not the messages you lose.

PARTITION 0



Ten slides to make it finally click. Swipe.

## THE WHOLE DIFFERENCE

# A queue deletes. A log keeps.

RabbitMQ delivers the message and wipes it. Kafka writes it down and leaves it lying there.

### QUEUE AFTER READING



### LOG AFTER READING



A3

group A • read up to here

A consumer moves only its own bookmark. The records stay where they are — that is why they can be read again.

# Every consumer has its own bookmark

PARTITION 0

LEO 0



RECORDS  
0

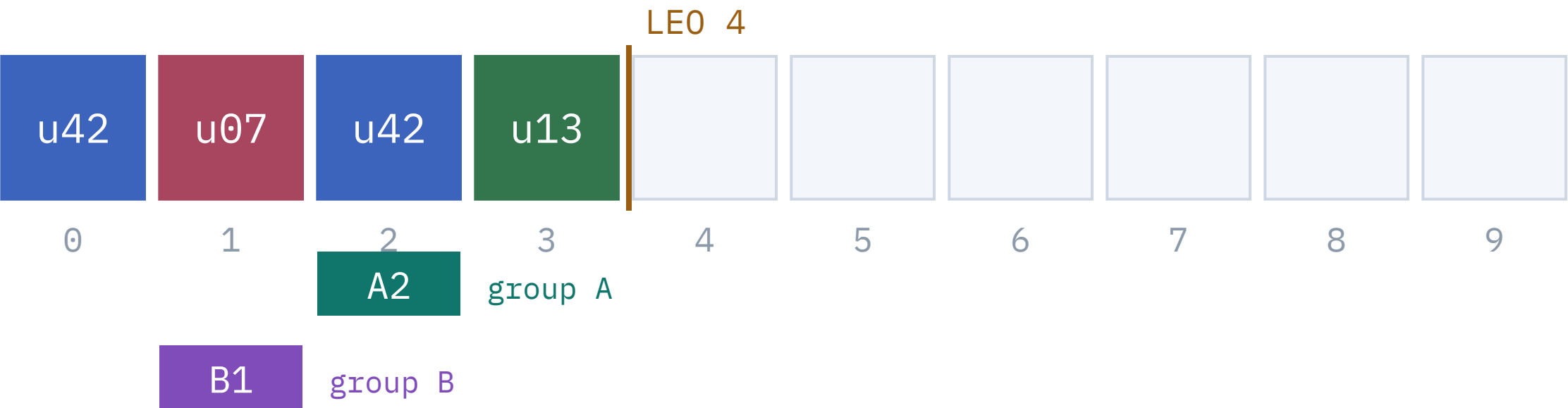
LAG OF GROUP A  
0

LAG OF GROUP B  
0

The log is empty. Both groups sit at offset 0 – there is nothing to read yet.

# The producer writes to the end

PARTITION 0



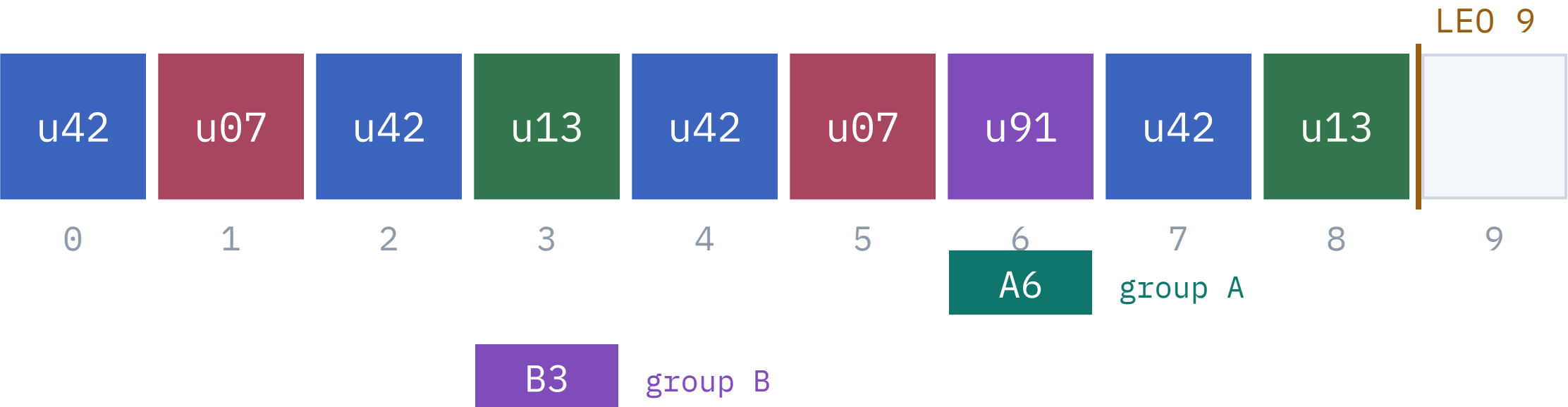
| RECORDS | LAG OF GROUP A | LAG OF GROUP B |
|---------|----------------|----------------|
| 4       | 2              | 3              |

The producer appended four records. The groups read at their own pace and have already drifted apart.

# This is what lag looks like

lag = end of the log minus the position of the group. The health metric that matters: how stale the data is right now.

PARTITION 0



|         |                |                |
|---------|----------------|----------------|
| RECORDS | LAG OF GROUP A | LAG OF GROUP B |
| 9       | 3              | 6              |

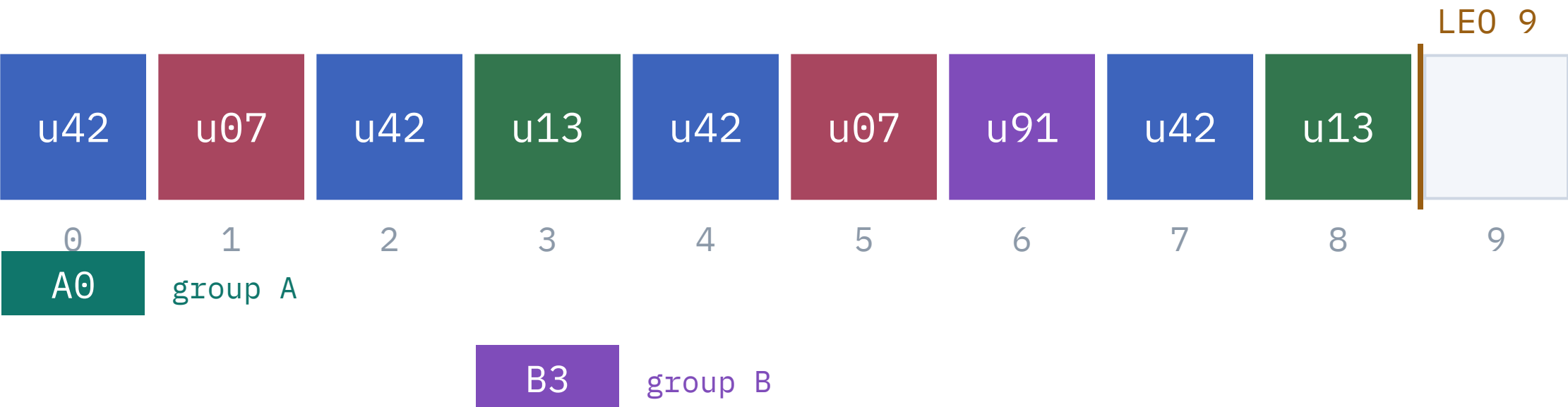
The lag is plain to see: group B has six unread messages piled up behind its bookmark.

WHAT A QUEUE CANNOT DO

# A bookmark can be rewound

The records never went anywhere, so the history gets read again – that is what replay means.

PARTITION 0

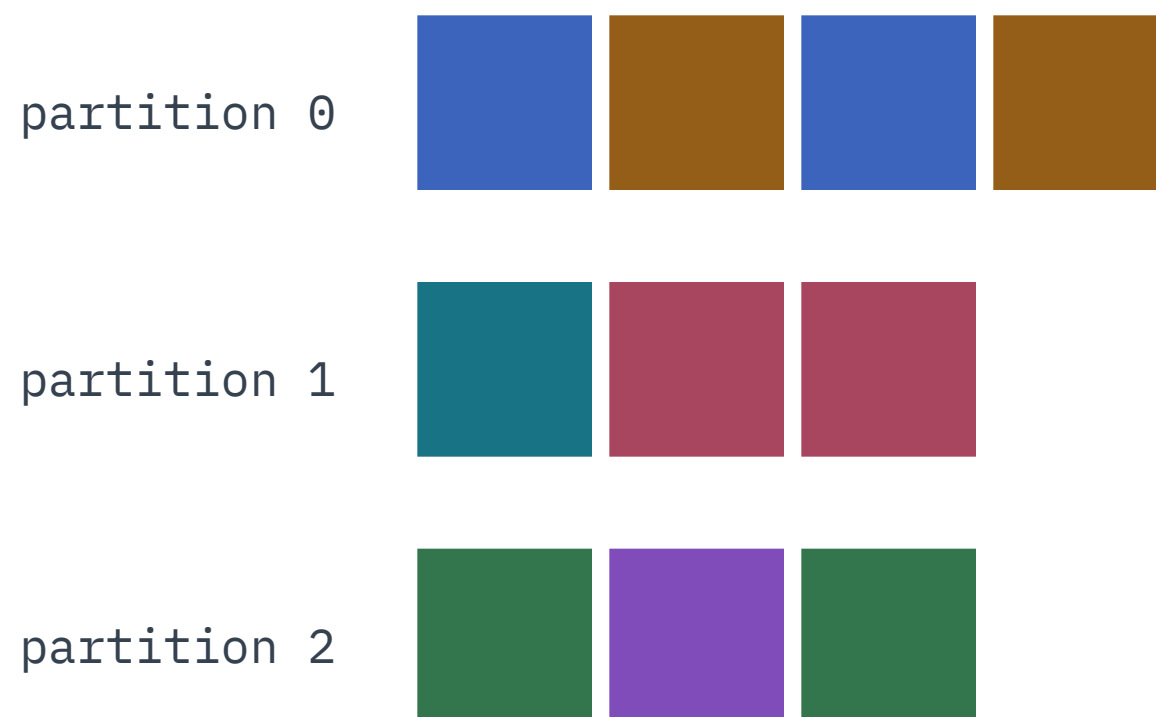


A new service joins a live topic and reads through everything that happened before it. A queue has no such button.

## THE LAW OF KAFKA

# The key decides where a record lands

partition number =  $\text{hash}(\text{key}) \% \text{number of partitions}$



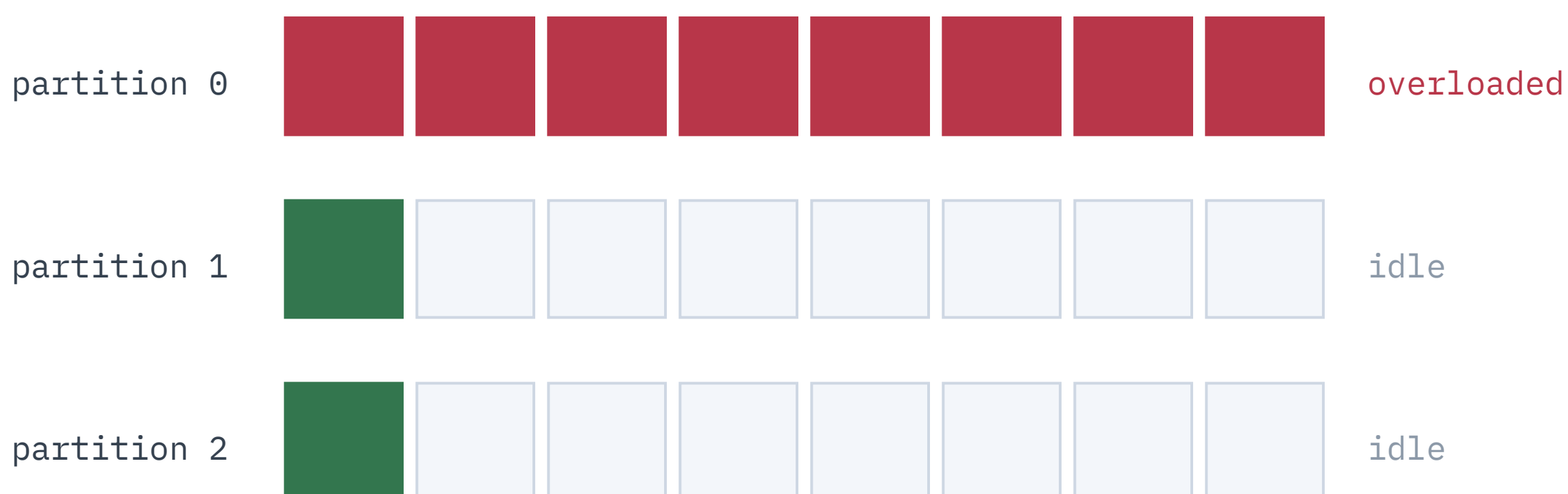
**same key → same partition → order**

Order is guaranteed inside a partition only. Across partitions there is no order at all: no shared counter, no shared clock.

## HOW IT BREAKS

# A bad key — and one partition burns

If eight events out of ten carry the same key, all eight land in one partition. It is overloaded, its neighbours idle.



Adding consumers will not help: inside a group one partition is read by exactly one of them. The ceiling is the partition count.

## THE MOST EXPENSIVE TRAP

# acks=all does not mean “all”

It waits for an acknowledgement from every replica IN THE ISR – the list of those keeping up with the leader. And that list can shrink.

### The replicas keep up

ISR = [1, 2, 3] · acks=all waits for three

**THE DATA IS SAFE**

### The replicas fell behind and dropped out

ISR = [1] · acks=all waits for ONE

**SILENT LOSS WHEN THE LEADER DIES**

### min.insync.replicas = 2

ISR = [1] · the write is refused

**AN OUTRIGHT ERROR – AND THAT IS GOOD**

The reliable trio, remembered as one thing: replication.factor 3, acks=all, min.insync.replicas 2.

AND NOW THE STRANGEST PART

# All of this is live. In a PDF.

The slides above are frames of real demos. A PDF has no HTML and no CSS, but it does have form fields and JavaScript of its own: the log cells are buttons, the motion is a timer, the captions are text fields.

Fifteen interactive chapters:

**[warhammer2000.github.io/kafka-na-palcah](https://warhammer2000.github.io/kafka-na-palcah)**

Runs in a browser, phones included.

The version with live demos inside the PDF is linked in the post text — download it and open it on a desktop, in Chrome, Edge or Acrobat.

Here in the feed it is all static: LinkedIn turns any document into images. So there are no buttons here — they are waiting in the file.